

Claude Code 从入门到精通学习指南

一、Claude Code 概览：是什么与为什么使用它？

Claude Code 的定位与优势

与其他工具的对比

Claude Code 能做什么？

二、快速入门：安装与基本使用

环境准备

安装 Claude Code

配置与首次运行

基本用法

三、熟练进阶：掌握 Claude Code 的工作原理与技巧

深入理解 Claude Code 的工作机制

CLAUDE.md：项目级配置与上下文记忆

权限与安全：信任与控制

常见工作流与应用技巧

四、专家精通：高级配置与扩展

扩展 Claude Code 的能力：MCP 与插件

自定义技能 (Skills)

自动化工作流：Hooks 的使用

代理团队与高级模式

持续优化与最佳实践

五、经典应用案例与配置示例

● Claude Code 概览：是什么与为什么使用它？

Claude Code 的定位与优势

Claude Code 是由 Anthropic 公司推出的一款**AI 编码助手**，它以命令行（CLI）形式存在，能够直接读取你的代码库、编辑文件、运行命令，并与其他开发工具集成。与传统的代码补全工具不同，Claude Code 更像是一个“代理式”（Agentic）助手：它会根据你提供的自然语言指令，自主规划并执行一系列操作，例如分析代码结构、修改多文件、运行测试等。这意味着你可以告诉它“帮我实现某某功能”，它会尝试理解整个项目并自动完成相应任务，而不仅仅是在光标位置给出代码片段。

Claude Code 的**优势**主要体现在以下几个方面：

- **深度代码库理解**：它能够索引和理解你的整个项目结构，知道文件之间的关系，因此在修改代码时可以做到跨文件的一致性。这使其非常适合处理大型代码库中的任务。
- **交互式推理**：Claude Code 在执行任务前会展示其思考过程，并在需要时询问你的意见。例如，它可能先给出一个计划，然后等待你确认再执行。这种透明度让你对它的行为有更好的掌控。
- **本地执行**：它直接在你的机器上操作文件和运行命令，而不是在远程沙盒中。这意味着它可以即时修改你的本地代码，但也需要你对它给予一定的信任。
- **权限控制**：出于安全考虑，Claude Code 默认对可能影响系统的操作（如修改文件、运行命令）需要你的批准。你可以精确地配置哪些工具允许自动执行，哪些需要人工确认。
- **多平台支持**：Claude Code 可以在终端、VS Code、JetBrains IDE、桌面应用甚至浏览器中使用。无论你偏好哪种开发环境，都可以无缝集成 Claude Code。

与其他工具的对比

Claude Code 与 GitHub Copilot 等 IDE 插件式代码补全工具最大的不同在于**作用范围和工作方式**。Copilot 等工具主要是在你编码时根据上下文提供代码片段或整行补全，而 Claude Code 则是像一个虚拟开发者一样，接受你描述的任务并尝试完成整个流程。例如，你可以让 Claude Code “写一个单元测试，然后运行它并修复失败”，它会自动执行一系列操作，而传统补全工具无法直接实现这样的复杂任务。此外，Claude Code 支持**多轮对话**，你可以与它进行多轮交流，逐步完善代码；而传统补全工具则更多是一次性的建议。总的来说，Claude Code 更适合处理**项目级别的复杂任务**，而 Copilot 等**擅长于编码过程中的即时辅助**。两者可以结合使用：先用 Claude Code 理清思路或处理繁重任务，再用 Copilot 进行细节编码。

Claude Code 能做什么？

Claude Code 的能力非常广泛，涵盖软件开发的多个方面。官方文档列出了它的一些核心用途：

- **自动化重复性任务**：例如为未测试的代码编写单元测试、修复整个项目的代码风格问题（lint 错误）、解决 Git 合并冲突、更新依赖版本、生成发布说明等。这些往往是你日常会拖延的杂务，Claude Code 可以帮你高效完成。
- **构建功能与修复缺陷**：你只需用自然语言描述想要的功能或遇到的问题，Claude Code 就会规划方案并编写代码。对于 Bug，你可以粘贴错误信息或描述现象，它会追踪问题根源并修复。
- **代码审查与提交**：Claude Code 可以直接与 Git 集成，自动暂存更改、撰写提交信息、创建分支甚至打开 Pull Request。你还可以在 CI/CD 中使用它来自动化代码审查和问题分流。
- **连接外部工具与数据**：通过 **Model Context Protocol MCP**，Claude Code 可以接入外部数据源和工具，例如读取 Google Drive 中的设计文档、更新 Jira 工单、从 Slack 拉取数据，甚至使用你自定义的工具。这大大扩展了它的应用范围，使其成为你工具链的一部分。
- **自定义扩展**：你可以通过创建**技能（Skills）**来赋予 Claude Code 新的能力。技能本质上是预定义的指令集，当满足某些条件时自动触发。例如，

你可以创建一个“代码审查”技能，让 Claude Code 在你提交代码前自动检查代码质量。技能支持团队共享，非常适合在企业环境中建立统一的开发规范。

- **团队协作与代理团队：** Claude Code 支持启动多个代理并行工作。一个“主代理”可以协调多个子代理，分别处理任务的不同部分，然后将结果合并。这对于大型任务非常实用，例如同时修改多个相关模块。

总而言之，Claude Code 将**对话式 AI** 引入了开发流程。它不仅能写代码，还能**理解上下文、规划步骤、执行操作**，并通过对话与你协作完成任务。这种模式有望将开发者的生产力提升到一个新层次。当然，前提是你需要掌握它的使用方法和最佳实践，这正是本指南接下来要探讨的内容。

● 快速入门：安装与基本使用

环境准备

在开始使用 Claude Code 之前，请确保你满足以下前提条件：

- **账户与订阅：** 你需要有一个有效的 Claude 订阅。通常，**Claude Pro** 订阅（每月 \$20）或更高的 **Anthropic Console** 账户即可满足使用要求。Claude Code 调用的是 Anthropic 的模型（如 Claude Opus 4.6 或 Sonnet 4.6），因此需要相应的权限和额度。
- **操作系统：** Claude Code 支持 macOS、Linux 和 Windows（建议通过 WSL2 使用）。本指南以 macOS/Linux 为例，Windows 用户建议使用 WSL 或 PowerShell 进行相应调整。
- **终端环境：** 确保你有一个可用的终端（如 macOS 的 Terminal 或 Linux 的终端）。对于 Windows，可以使用 PowerShell 或 WSL 的终端。
- **Node.js 环境（可选）：** 早期版本的 Claude Code 通过 npm 安装，需要 Node.js 环境。但官方现已提供独立安装包，不再强制依赖 Node.js。因此，Node.js 环境是**可选的**，如果你使用官方安装脚本，则无需提前安装 Node.js。

- **网络访问**：Claude Code 需要联网进行身份验证和模型调用。如果你所在地区无法直接访问 Anthropic 的服务，需要提前准备好网络环境或代理方案。

安装 Claude Code

Claude Code 提供了多种安装方式，官方推荐使用**原生安装**（Native Install）以获得自动更新功能：

1. **官方安装脚本（推荐）**：打开终端，执行以下命令来安装 Claude Code：

```
curl -fsSL https://claude.ai/install.sh | bash
```

该脚本会自动下载并安装适用于你操作系统的 Claude Code 可执行文件，并配置后台自动更新。安装完成后，你可以在任意项目目录下运行

`claude` 命令。

2. **通过包管理器安装**：如果你使用 macOS 并安装了 Homebrew，也可以通过 Homebrew Cask 安装：

```
brew install --cask claude-code
```

注意，通过 Homebrew 安装不会自动更新，你需要定期运行 `brew upgrade claude-code` 来获取最新版本。类似地，Windows 用户可以使用 `winget install Anthropic.ClaudeCode` 进行安装，同样需要手动更新。

3. **NPM 全局安装（旧方法）**：在官方提供独立安装包之前，Claude Code 曾通过 npm 发布。如果你之前使用 npm 安装过，可以运行以下命令进行迁移：

```
claude install
```

该命令会下载新的原生可执行文件并替换旧的 npm 版本。然后，建议卸载旧的 npm 包：

```
npm uninstall -g @anthropic-ai/claude-code
```

以确保没有残留。这种方法现已不推荐，除非你出于特殊原因需要使用 npm 版本。

安装完成后，你可以通过运行 `claude --version` 来验证安装是否成功，以及查看当前版本号。如果出现“command not found”错误，请检查你的 PATH 环境变量是否包含 Claude Code 可执行文件所在的路径。

配置与首次运行

身份验证：首次运行 Claude Code 时，它会提示你进行登录认证。Claude Code 支持多种认证方式，包括使用 Anthropic 的账户、GitHub 账户等。按照提示完成登录即可。这一步将获取一个访问令牌（API Key）并保存，用于后续的模式调用。

API Key 配置（可选）：如果你已经拥有 Anthropic 的 API Key，并希望手动配置，可以创建或编辑配置文件 `~/.claude/settings.json`（用户全局配置）或项目目录下的 `.claude/settings.json`（项目级配置）。在配置文件中添加如下内容：

```
{
  "env": {
    "ANTHROPIC_API_KEY": "你的API密钥"
  },
  "permissions": {
    "allow": [],
    "deny": []
  }
}
```

这样，Claude Code 将使用你提供的 API Key 进行调用。请注意，`ANTHROPIC_API_KEY` 是一个环境变量，Claude Code 会读取它。你也可以通过命令行参数 `--api-key` 临时指定 API Key，但将其保存在配置文件中更为方便。

选择工作目录： Claude Code 最好在项目根目录下运行，这样它能全面了解你的代码库。例如：

```
cd ~/projects/my-app
claude
```

这将启动 Claude Code 并索引当前项目的文件结构。如果你的项目非常大，首次启动可能需要一些时间来建立索引。

常用命令速览： 进入 Claude Code 的交互式界面后，你可以使用一些内置命令来辅助使用：

- `/help`：显示所有可用的命令和技能。这些命令都以 `/` 开头。
- `/plan`：进入**规划模式**，让 Claude Code 先制定详细计划再执行。这在处理复杂任务时非常有用，稍后会详细介绍。
- `/clear`：清空当前对话上下文，重新开始。当你想开始一个新的任务时，可以使用此命令，以免之前的对话干扰。
- `/cost`：查看当前会话已使用的令牌数和预估费用。
- `/compact`：压缩对话历史，以节省上下文窗口空间。当对话很长时，可以使用此命令让 Claude Code 总结之前的对话内容，从而腾出“记忆空间”。

以上命令不需要记忆，你只需记住输入 `/` 后跟关键字即可调用。在日常使用中，大多数情况下你直接用自然语言提问或下达指令即可，Claude Code 会尽力理解并执行。

基本用法

启动 Claude Code 后，你会进入一个类似于对话的界面。你可以在提示符后输入问题或指令，Claude Code 会根据你的输入进行回应或操作。下面是一个简单的交互示例：

> 请总结一下这个项目的技术栈和目录结构。

Claude Code 会扫描你的项目文件，并给出一个结构化的回复，包括使用的框架、主要模块及其作用等。接下来，你可以尝试让它执行一个实际任务：

> 为 signup 表单添加输入验证。邮箱需要验证格式，密码至少8个字符。

Claude Code 将会：

1. **查找相关代码：**定位到注册表单组件或相关验证逻辑的文件。
2. **展示计划：**如果开启了规划模式，它会先列出将要进行的更改（例如修改哪些文件，添加哪些验证函数）。
3. **请求许可：**在真正修改文件前，会询问你是否允许修改这些文件。
4. **应用更改：**得到许可后，它会编辑文件，添加邮箱格式验证和密码长度检查等代码。
5. **确认完成：**最后，它会总结做了哪些修改，并可能建议运行测试或查看更改。

整个过程你可以通过对话界面看到 Claude Code 的思考和操作步骤。这种**透明可控**的方式让你对 AI 的行为有清晰了解，也更容易信任它。

重要提示：Claude Code 默认对敏感操作（如修改文件、运行命令）保持谨慎。它会先征求你的同意再执行。这是一种安全机制，防止它误操作你的代码。你可以通过权限配置来调整这一点，稍后我们会在进阶部分详细讲解。在刚开始使用时，建议保持默认设置，多观察它的行为，逐步建立信任。

通过以上步骤，你已经完成了 Claude Code 的基本入门。接下来，我们将深入探讨 Claude Code 的工作原理，以及如何更高效地使用它。

深入理解 Claude Code 的工作机制

要熟练使用 Claude Code，你需要了解其**上下文管理**和**规划机制**。Claude Code 的强大来自于它对项目上下文的掌握和对任务的规划能力。

上下文管理：Claude Code 通过读取你的代码文件来建立项目的“记忆”。它会对项目进行索引，理解文件之间的依赖关系。当你提出一个请求时，它会尝试定位相关的文件并在上下文中加载这些文件的内容。然而，模型的**上下文窗口**是有限的（例如 200K token，相当于数十万字符的代码），因此如何高效利用上下文就非常关键。如果项目非常大，一次性加载所有文件是不可能的。Claude Code 会根据任务需要，动态地读取相关文件。这意味着，如果你在对话中频繁切换话题或涉及不相关的内容，上下文可能被稀释，导致 Claude Code “忘记”之前的细节。

为了解决上下文管理的问题，有经验的用户会采取一些策略：

- **定期清理上下文：**使用 `/clear` 命令清空对话，以防止之前的无关信息干扰当前任务。
- **使用规划模式：**在规划模式下，Claude Code 会先分析问题并给出计划，但不会立即执行。这有助于在上下文中保留**规划信息**，而不是立即被大量的文件内容淹没。
- **CLAUDE.md 项目说明文件：**通过在项目根目录放置一个 `CLAUDE.md` 文件，你可以提供项目的**元信息**，如技术栈、编码规范、测试命令等。Claude Code 在每次会话开始时会自动读取这个文件，从而预先了解项目的基本情况，而不需要在对话中反复询问或猜测。

规划机制：Claude Code 在执行复杂任务时，通常会经历一个“**思考-规划-执行**”的过程。它会先思考任务的步骤，然后制定计划，最后执行。例如，当你要求重构一个模块时，它可能会：

1. **分析：**阅读相关代码，理解模块功能和依赖。
2. **规划：**列出需要修改的文件、新增的函数、需要更新的调用等。

3. **执行**：按照计划逐步修改代码。

这种规划机制可以通过**规划模式（Plan Mode）**显式触发。在规划模式下，Claude Code 会**暂停执行**，只展示计划和理由，等待你确认后再执行。规划模式非常适合处理**高风险或复杂**的任务，例如：

- **大规模重构**：涉及多个文件、模块的修改。
- **多文件功能**：新增一个功能，需要同时修改前后端代码、测试等。
- **陌生代码库**：当你在不熟悉的代码库中工作时，规划模式可以让你先理解影响范围再动手。

启用规划模式很简单，只需在对话中输入 `/plan`，然后给出任务描述即可。例如：

```
> /plan
> 将认证模块从使用 session 改为使用 JWT。
```

Claude Code 会响应类似以下内容：

```
Here is my plan:
1. 更新 auth/middleware.js, 验证 JWT 而非 session。
2. 修改 auth/login.js, 生成并返回 JWT。
3. 更新 auth/logout.js, 处理 JWT 失效。
4. 调整前端 API 客户端, 发送 Authorization 头。
5. 更新测试文件以使用 JWT 认证。
是否继续？
```

你可以审查这个计划，询问细节，甚至要求它调整方案。确认后，再让它执行。规划模式确保“三思而后行”，避免“一通乱改”带来的错误。

CLAUDE.md：项目级配置与上下文记忆

正如前面提到的，`CLAUDE.md` 是一个特殊文件，用于提供项目的**静态上下文**。你可以把它看作是给 Claude Code 的“项目说明书”。在 Claude Code

启动时，它会自动读取项目根目录下的 `CLAUDE.md` 文件（如果存在）。这个文件通常采用 Markdown 格式，包含以下内容：

- **项目概述**：项目名称、主要功能和用途。
- **技术栈**：使用的编程语言、框架、库。
- **编码规范**：代码风格、命名约定、是否使用 TypeScript 等。
- **测试与构建**：如何运行测试、构建项目、常用命令。
- **Git 规范**：提交信息格式（如 Conventional Commits）、分支命名等。
- **团队成员**：可选，列出主要开发者，以便 Claude Code 了解人员分工。

例如，一个 `CLAUDE.md` 示例可能如下：

```
# Project: My SaaS App

## Tech Stack
- Next.js 14 with App Router
- TypeScript (strict mode)
- Tailwind CSS
- PostgreSQL with Prisma ORM

## Coding Conventions
- Use functional components with hooks (no class components)
- All functions must have TypeScript return types
- Use named exports, not default exports
- Write tests for all new utility functions

## Testing
- Run tests with: `npm run test`
- Use Vitest for unit tests
- Test files go in `__tests__` directories next to source file

## Git
- Conventional commits: `feat()`, `fix()`, `refactor()`
- Always create a new branch for features
```

有了这个文件，Claude Code 在后续对话中就会遵循这些约定。例如，它会默认使用函数式组件，而不是生成类组件；它会知道运行测试用 `npm run`

`test` 等。这大大减少了反复沟通的需要，提高了效率。

最佳实践：将 `CLAUDE.md` 放入版本控制，让整个团队共享。这意味着每个人在使用 Claude Code 时都会有一致的上下文。对于大型组织，甚至可以有**企业级的 CLAUDE.md**，放在所有项目共享的父目录，用于定义通用的编码规范和流程。Claude Code 会合并不同层级的 `CLAUDE.md` 文件（如果存在多个），以提供全面的上下文。

权限与安全：信任与控制

Claude Code 默认采用**保守的权限策略**，即需要用户确认才会执行可能修改系统的操作。这种设计是出于安全考虑，防止 AI 在未经许可的情况下对代码库造成破坏。然而，随着你对 Claude Code 的信任加深，频繁的确权可能会降低效率。因此，Claude Code 提供了**权限配置**来让你精细控制哪些操作可以自动执行。

权限配置主要涉及两类：**工具权限**和**文件权限**。

工具权限：Claude Code 内部定义了一系列“工具”，例如 `Edit`（编辑文件）、`Write`（写入文件）、`Bash`（运行 shell 命令）等。你可以指定哪些工具允许 Claude Code 自动使用，哪些需要手动批准。例如，你可能信任它运行某些测试命令，但不允许它修改生产配置文件。

在配置文件 `.claude/settings.json` 中，可以通过 `permissions.allow` 和 `permissions.deny` 来设置。例如：

```
{
  "permissions": {
    "allow": ["Bash npm test", "Bash npm run lint"],
    "deny": ["Write src/config/*"]
  }
}
```

上述配置表示：允许 Claude Code 自动运行 `npm test` 和 `npm run lint` 命令，但禁止它写入 `src/config/` 目录下的任何文件。你可以根据项目需要，灵活配置允许和拒绝的工具列表。

文件权限：除了工具级别的权限，你也可以指定**额外的工作目录**。默认情况下，Claude Code 只能访问你启动它的项目目录及其子目录。但你可以通过配置，让它读取或修改项目外的一些目录。例如：

```
{
  "permissions": {
    "additionalDirectories": ["../docs/", "../shared/"]
  }
}
```

这样，Claude Code 可以访问上级目录下的 `docs` 和 `shared` 文件夹，这对于多模块项目或需要引用外部文档的情况很有用。

审计日志：Claude Code 还提供审计日志功能，记录它执行的所有操作。你可以通过日志来回顾它做了哪些修改，这也是建立信任的一种方式。在企业环境中，审计日志有助于满足合规要求。

总的来说，权限配置让你在**效率和安全**之间取得平衡。刚开始使用时，建议保持默认的“每次确认”模式，观察 Claude Code 的行为。随着信任的建立，再逐步放开一些权限，提高自动化程度。

常见工作流与应用技巧

掌握了基本原理和配置后，下面介绍几个常见的工作流和应用技巧，帮助你更高效地使用 Claude Code。

1. 提交代码与创建 PR

Claude Code 可以直接与 Git 集成，帮你完成代码提交和 PR 创建。例如，当你完成一系列修改后，可以告诉它：

```
> commit my changes with a descriptive message
```

Claude Code 会：

1. **暂存更改**：使用 `git add` 将所有相关更改添加到暂存区。
2. **撰写提交信息**：根据修改内容生成一个描述性的提交信息。
3. **执行提交**：运行 `git commit`。

如果需要创建 Pull Request，可以进一步说：

```
> create a PR for this commit
```

它会：

1. **创建分支**：如果当前不在分支上，可能会新建一个分支。
2. **推送到远程**：将提交推送到 GitHub 等远程仓库。
3. **创建 PR**：使用提交信息作为 PR 标题，打开一个 Pull Request。

当然，这些操作都需要你在 `CLAUDE.md` 中提供相应的 Git 规范（如分支命名、提交信息格式），以便 Claude Code 遵循。

2. 自动化测试与持续集成

Claude Code 可以成为你 CI/CD 流程的一部分。例如，在 GitHub Actions 中，你可以配置一个工作流，在每次代码推送时，让 Claude Code 运行代码审查或测试。这类似于使用一个 AI 助手来做**静态代码分析**或**测试辅助**。

一个简单例子是在每次 Pull Request 时，让 Claude Code 检查代码风格和生成测试：

```
name: Claude Code Review
on: pull_request
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run Claude Code
```

```
run: |  
  claude -p "Review this pull request for code quality"
```

这只是一个示意，实际使用时需要考虑如何传递上下文（例如使用 GitHub Token 让 Claude Code 获取 PR 的信息）。

3. 团队协作与代理团队

对于大型任务，Claude Code 支持启动多个**子代理**并行工作。你可以让一个“主代理”协调多个“子代理”，分别处理任务的不同部分。例如：

```
> /team  
> 将 src/ 目录下的所有组件从 Solid 迁移到 React。
```

Claude Code 会分析任务，将其拆分为多个独立的单元，然后并行启动多个代理，每个代理在一个隔离的 Git 工作树中执行一部分迁移。完成后，它会汇总结果并可能创建多个 Pull Request。这种**并行处理**能力非常适合大规模重构或迁移任务。

4. 使用 MCP 获取外部数据

假设你在开发过程中需要参考一个 Jira 票或 Google Drive 文档。你可以通过 MCP 让 Claude Code 获取这些信息。例如，如果你有一个 Jira 服务器 MCP 插件，你可以这样问：

```
> 查询 Jira 票号 PROJ-123，并根据其描述生成测试用例。
```

Claude Code 会通过 MCP 调用 Jira，获取票务信息，然后根据描述生成相应的测试代码。这展示了 Claude Code 与外部工具集成的强大之处。

5. 自定义技能

你可以创建**技能（Skills）**来封装常用的工作流。例如，创建一个“部署到 staging”技能，当你输入 `/deploy-staging` 时，Claude Code 会执行一系列预设的命令（如构建、推送镜像、通知等）。技能可以是简单的

Markdown 文件，放在项目的 `.claude/skills/` 目录下。随着你使用深入，不妨尝试创建自己的技能，提高团队的开发效率。

以上只是 Claude Code 应用场景的冰山一角。随着你对它理解的加深，你会发现它几乎可以融入开发的各个环节。关键在于**不断尝试**，并根据自己的需求调整使用方式。

● 专家精通：高级配置与扩展

当你已经成为 Claude Code 的熟练用户后，可以进一步探索其高级功能，包括**模型选择**、**多代理协作**、**自定义扩展**等，以达到专家级的精通水平。

扩展 Claude Code 的能力：MCP 与插件

Claude Code 提供了丰富的扩展机制，使其能够适应各种开发需求。

Model Context Protocol MCP 是一个开放标准，用于将 Claude Code 与外部数据源和工具连接起来。通过 MCP，Claude Code 可以：

- 读取存储在外部系统的设计文档、需求文档（例如 Google Drive、Confluence）。
- 与项目管理工具交互（例如 Jira、Trello），自动更新任务状态。
- 调用自定义的工具或脚本，扩展 Claude Code 的能力边界。

要使用 MCP，你需要配置相应的 MCP 服务器。这通常涉及在

`settings.json` 中添加 MCP 服务器的配置，包括服务器 URL、认证信息等。一旦配置完成，Claude Code 就能在需要时自动调用这些外部资源。

插件（Plugins） 是 Claude Code 提供的另一层扩展机制。插件可以看作是**打包好的扩展模块**，可以包含技能、MCP 配置、代理定义等。Anthropic 提供了一个插件市场，你可以发现并安装预构建的插件，例如代码格式化插件、团队协作插件等。你也可以创建自己的插件，供团队或社区使用。

自定义技能 (Skills)

自定义技能是 Claude Code 高级用法的重要组成部分。通过创建技能，你可以赋予 Claude Code 新的行为，让它更好地服务于你的项目和工作流程。

创建技能非常简单：

1. **创建目录**：在项目的 `.claude/skills/` 目录下新建一个文件夹，例如 `my-skill`。
2. **编写 SKILL.md**：在 `my-skill` 文件夹下创建一个 `SKILL.md` 文件。这个文件包含两大部分：
 - **YAML 前言**（在文件开头，用 `---` 包围）：用于定义技能的名称、描述、触发条件等元数据。
 - **指令内容**（在 YAML 之后）：用 Markdown 编写的具体指令，告诉 Claude Code 在该技能触发时应该做什么。

例如，创建一个“代码解释”技能：

```
---
name: explain-code
description: 使用类比和图示解释代码的工作原理。当用户询问“这段代码如何工
---
在解释代码时，请遵循以下步骤：
1. **用一个日常生活中的类比**开始，帮助理解。
2. **绘制一个简单的 ASCII 流程图**，展示代码的执行流程。
3. **逐行讲解代码**的作用。
4. **指出常见的陷阱或误区**，提醒注意。
```

将上述内容保存为 `~/.claude/skills/explain-code/SKILL.md`（个人技能）或项目目录下的相应位置。这样，当你在对话中问“这段代码如何工作？”时，Claude Code 就会自动调用这个技能，按照你定义的格式来解释代码。

技能的进阶用法：

- **支持文件**：你可以在技能目录下放置模板文件、示例输出文件、脚本等。在 `SKILL.md` 中引用它们，让 Claude Code 根据模板填充内容，或执行 

脚本。

- **动态上下文注入**：技能可以定义在特定事件（如每次会话开始）时自动注入一些上下文。这类似于 CLAUDE.md，但更灵活，可以基于条件触发。
- **子代理执行**：技能可以指定 Claude Code 在执行某些步骤时使用子代理。例如，一个“代码审查”技能可能让 Claude Code 启动一个子代理来审查代码，然后将结果合并。
- **限制访问**：你可以控制哪些用户或项目可以使用某个技能。

通过自定义技能，你可以将团队的最佳实践固化下来，让 Claude Code 成为团队的“数字同事”。

自动化 workflow：Hooks 的使用

Hooks 是 Claude Code 提供的自动化机制，允许你在特定事件发生时自动执行 shell 命令。这可以用来**格式化代码**、**发送通知**、**验证操作**等。

Hook 的触发时机有很多，常见的包括：

- **PostToolUse**：在 Claude Code 使用某个工具之后。例如，每次它编辑完文件后，你可以自动运行格式化工具。
- **PreToolUse**：在 Claude Code 使用某个工具之前。例如，在它尝试运行某个命令前，你可以检查环境是否满足条件。
- **Notification**：当 Claude Code 需要你的输入或权限时。你可以设置一个 hook，在此刻发送桌面通知，提醒你回来查看。

配置 Hook 非常简单。你可以使用交互式菜单 `/hooks` 来创建，也可以直接编辑 `settings.json`。例如，下面是一个自动格式化代码的 Hook 配置：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
```

```

        "type": "command",
        "command": "jq -r '.tool_input.file_path' | xargs
    }
]
}
]
}
}

```

这段配置的含义是：每当 Claude Code 使用了 `Edit` 或 `Write` 工具（即修改了文件），就运行 `npx prettier --write` 对该文件进行格式化。通过这种方式，你可以确保代码风格始终一致，无需每次提醒 Claude Code 去运行格式化工具。

再举一个通知的例子。如果你希望在 Claude Code 完成任务或需要你确认时收到通知，可以配置一个 Notification Hook：

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Claude Code
          }
        ]
      }
    ]
  }
}

```

这样，每当 Claude Code 有重要事项需要你关注时，你都会收到一个桌面通知。

Hooks 的强大之处在于你可以**编写脚本**来实现复杂的逻辑。例如，你可以编写一个脚本，在每次提交前自动检查代码是否包含敏感信息，如果包含则阻止提

交。这将极大增强 Claude Code 的安全性和可控性。

代理团队与高级模式

Claude Code 提供了一些高级模式，适合处理复杂任务或大规模项目。

代理团队 (Agent Teams)：如前所述，你可以启动多个 Claude Code 代理并行工作。这非常适合大型重构、迁移等任务。你可以手动使用 `/team` 命令，也可以让 Claude Code 在需要时自动分解任务。代理团队的使用需要你对任务有清晰的划分，并信任 Claude Code 能够独立完成子任务。

计划模式 (Plan Mode)：这是已经介绍过的一个重要模式，值得反复强调。对于任何复杂任务，都建议先进入计划模式，让 Claude Code 展示其思考过程。这不仅能帮助你理解它的思路，也能在出现问题时及时调整。在专家级使用中，你应该**习惯于先规划，再执行**，以避免不必要的错误。

上下文压缩：当对话非常长时，你可以使用 `/compact` 让 Claude Code 总结之前的对话。这会压缩上下文，但保留关键信息。在需要长时间对话的场景下，这可以防止上下文溢出。

模型选择：Claude Code 默认使用 Anthropic 最新的模型（如 Claude Opus 4.6 或 Sonnet 4.6）。这些模型有不同的特点：Opus 模型能力更强，适合处理复杂任务，但响应可能稍慢；Sonnet 模型则在速度和能力上取得平衡。你可以根据任务需要在配置中指定模型，或者在对话中通过 `/model` 命令临时切换模型。

持续优化与最佳实践

成为 Claude Code 专家，需要你在实践中不断总结和优化。以下是一些高级最佳实践：

- **持续更新 CLAUDE.md：**项目是不断演进的，定期更新 `CLAUDE.md`，将新的约定、依赖、脚本命令等信息告诉 Claude Code。
- **定期审查权限和 Hooks：**随着项目变化，你可能需要调整权限配置或 Hooks。例如，新增了一个敏感目录，需要在 `permissions.deny` 中添

加；或者引入了新的工具，需要配置相应的 Hook。

- **学习社区经验：** Claude Code 有一个活跃的社区。关注社区分享的技巧、技能和最佳实践，不断丰富你的知识库。
- **组合使用多种工具：** 不要孤立地使用 Claude Code。将它与你现有的开发工具链结合，例如代码编辑器、CI/CD、项目管理工具等，形成一套高效的开发流程。

通过以上高级技巧的掌握，你已经从入门走到了精通。Claude Code 将不再只是一个工具，而会成为你开发工作中的**合作伙伴**，帮助你以更高的效率和质量完成项目。

● 经典应用案例与配置示例

为了进一步巩固所学，下面提供几个经典的应用案例和相应的配置示例。这些案例展示了 Claude Code 在不同场景下的用法，你可以根据自己的需求进行调整。

案例一：自动代码格式化与风格检查

场景： 你希望每次 Claude Code 修改代码后，都自动运行代码格式化工具（如 Prettier）和代码风格检查工具（如 ESLint），以确保代码风格一致。

解决方案： 使用 PostToolUse Hook 来触发格式化和检查。配置示例如下：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
```

```

        "type": "command",
        "command": "jq -r '.tool_input.file_path'
    },
    {
        "type": "command",
        "command": "jq -r '.tool_input.file_path'
    }
]
}
]
}
}

```

效果：每当 Claude Code 修改了文件，都会自动运行 Prettier 格式化代码，然后运行 ESLint 并尝试自动修复问题。如果 ESLint 发现无法自动修复的问题，会在终端输出警告，你可以及时查看。

案例二：保护敏感文件不被修改

场景：你的项目中有一些敏感文件，如 `.env`（环境变量）、`package-lock.json`、`.git/` 目录等，你不希望 Claude Code 修改它们。

解决方案：使用 PreToolUse Hook 来阻止对敏感文件的操作。你需要编写一个脚本，检查目标文件路径是否匹配敏感模式，如果匹配则返回错误码，阻止操作。配置示例如下：

```

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/protect-files.sh"

```

```

    }
  ]
}
]
}
}
}

```

protect-files.sh 脚本内容示例：

```

#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path')
PROTECTED_PATTERNS=".env" "package-lock.json" ".git/"
for pattern in "${PROTECTED_PATTERNS[@]}; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo "Blocked: $FILE_PATH matches protected pattern"
    exit 2
  fi
done
exit 0

```

效果：每当 Claude Code 尝试写入一个文件时，都会先调用这个脚本。如果文件路径匹配敏感模式，脚本会返回非零状态码，Claude Code 会收到反馈，知道该操作被阻止，并会尝试其他方案。

案例三：团队级代码审查技能

场景：你希望在每次提交代码前，Claude Code 都能自动执行一套代码审查流程，包括检查代码复杂度、查找潜在 bug、验证测试覆盖率等。

解决方案：创建一个“代码审查”技能，让 Claude Code 在提交前自动运行。技能内容可以包括：

- 运行代码复杂度分析工具，如 `eslint --rule 'complexity: ["error", 10]` 。
- 运行测试覆盖率工具，如 `npm run test -- --coverage` 。
- 使用 Claude Code 的分析能力，检查代码是否遵循团队的编码规范（这些规范已写入 CLAUDE.md）。

技能示例（ SKILL.md ）：

```
---
name: code-review
description: 在提交代码前执行自动化的代码审查。检查代码质量、测
---
```

在用户提交代码前，请执行以下步骤：

1. 运行 ``npm run lint`` 检查代码风格。
2. 运行 ``npm run test -- --coverage`` 检查测试覆盖率。
3. 分析代码变更，确保遵循项目编码规范（参考 CLAUDE.md）。
4. 如果发现任何问题，列出问题清单并建议修复方案。

效果：每当你提交代码，可以运行 `/code-review`。Claude Code 会自动执行上述检查，并给出报告。如果发现问题，你可以及时修复，从而在代码合并前就保证质量。

案例四：跨项目任务自动化

场景：你的公司有多个项目，都需要执行类似的部署流程。你希望有一个统一的 Claude Code 技能，能够在不同项目中执行部署。

解决方案：在企业级配置中创建一个共享技能，放置在所有项目都能访问的父目录 `.claude/skills/` 下。技能内容可以包括：

- 识别当前项目类型（例如，通过读取 `package.json` 或其他配置文件）。
- 根据项目类型，执行相应的部署脚本或命令。

- 在部署完成后，发送通知到团队 Slack 频道。

技能示例（`deploy.md`）：

```
---
name: deploy
description: 将当前项目部署到生产环境。适用于公司所有项目。
---

执行部署：
1. 检查当前项目是否为 Node.js 应用（存在 package.json）。
2. 如果是 Node.js 应用，运行 `npm run build`。
3. 将构建产物部署到服务器（假设有脚本 `deploy.sh`）。
4. 部署完成后，使用 Slack Webhook 发送通知到 #deployments
```

效果：无论你在哪个项目目录下，只需输入 `/deploy`，Claude Code 都会根据项目类型执行相应的部署流程，并在完成后通知团队。这大大简化了跨项目部署的流程，提高了团队协作效率。

以上案例只是冰山一角。通过灵活运用 Claude Code 的各项功能，你可以创造出无数种用法。关键在于**明确你的需求**，然后思考如何利用 Claude Code 提供的工具（对话、规划、技能、Hook、MCP 等）来实现它。随着实践的深入，你会发现 Claude Code 正逐渐成为你开发工作中不可或缺的伙伴。

● 总结

Claude Code 是一款革命性的 AI 编码助手，它以对话为核心，将 AI 深度融入开发流程。从入门到精通，你需要理解其工作原理，掌握基本用法，并逐步探索高级功能和扩展机制。本指南通过系统化的讲解和案例示范，旨在帮助你建立起完整的学习路径。

在实际使用中，以下几点值得牢记：

- **上下文是关键：**通过 `CLAUDE.md` 和规划模式，为 Claude Code 提供清晰的上下文，才能充分发挥其能力。

- **信任与控制并重：**逐步建立对 Claude Code 的信任，但始终通过权限配置保持必要的控制。
- **持续学习与实践：**关注官方文档和社区动态，不断尝试新功能，将 Claude Code 融入你的日常工作。

Claude Code 的出现，标志着 AI 在软件开发中的角色正从辅助工具向协作伙伴转变。掌握它，你将站在时代的前沿，以更高的效率和创造力迎接未来的挑战。祝你在 Claude Code 的世界里探索愉快！